

MicroK8s Scalable Microservices Project

Cloud Computing — Tamkang University | 2026

吳得豐 412855180 · 林大偉 412855263 · 高曼柯 412855289 · 陳明義 412855339

1. Introduction

This project deploys a stateless Flask microservice on a single-node MicroK8s cluster running inside a QEMU Ubuntu VM with constrained hardware (4 GB RAM, 1 vCPU). The goal was to demonstrate that core cloud-native capabilities — automatic scaling, self-healing, and traffic routing — are achievable on minimal infrastructure.

2. Architectural Choices

2.1 Application Design:

The Flask application is stateless, exposing four endpoints:

- **GET /** (HTML pod metadata),
- **GET /info** (ASCII-art plain-text response),
- **GET /health** (liveness probe), and
- **GET /ready** (readiness probe).

Pod identity (hostname and IP) is injected at runtime via the Kubernetes Downward API as the `POD_IP` environment variable to enable real-time traffic distribution to be visible in the UI without any code changes.

2.2 Containerization Strategy

Uses **python:3.9-slim** as the base image to reduce the final image size to 180 MB. Layer caching ensures dependencies only rebuild when `requirements.txt` changes. Images are stored in the MicroK8s built-in registry at `localhost:32000`, eliminating all external network dependency during deployment.

2.3 Kubernetes Orchestration

The deployment uses four manifest files located in `k8s/`:

- **Deployment**: manages pod lifecycle with a baseline of 3 replicas.
- **Horizontal Pod Autoscaler (HPA)**: scales between 3 and 10 replicas based on average CPU utilization, with a 50% target.
- **Ingress**: via Traefik, routes external requests to `flask-app.local`, with SSH tunneling from the Windows host used to bridge the QEMU network boundary.
- **Service**: a ClusterIP Service that provides internal load balancing.

2.4 Self-Healing Mechanisms

Two probes are configured per pod:

- The **liveness probe** (**GET /health**, every 10 s after a 15 s initial delay) detects stuck or crashed containers and triggers an automatic restart, recovering within ~20 seconds.
- The **readiness probe** (**GET /ready**, every 5 s after a 5 s delay) gates traffic: a pod that fails readiness is removed from the Service endpoint slice but is not deleted.

When a pod is deleted on purpose, Kubernetes automatically creates a new one within seconds. Traffic keeps going to healthy pods with no service interruption.

3. Hardware Constraints & Resource Allocation

Each pod is configured with a **64 Mi / 50m** request and a **256 Mi / 200m** limit:

Metric	Idle (3 pods)	Under Load (6–10 pods)	Node Hard Limit
CPU Usage	2–5%	~30–65%	1 vCPU (1000m)
Memory (actual)	90–150 Mi	~300–600 Mi	4096 Mi
CPU reserved (requests)	150m (3 × 50m)	Up to 500m (10 × 50m)	—
Memory reserved (requests)	192 Mi (3 × 64 Mi)	640 Mi (10 × 64 Mi)	—
Memory % of node	~4.8%	~15.6%	100%
Network	< 1 Mbps	~5 Mbps	—

A 4x Request-to-Limit ratio provides high burst capacity during traffic spikes, allowing the system to temporarily consume more resources up to their limits without risking node instability.

4. Limitations and Testing Observations

- HPA Behavior and the Stabilization Window Problem:
We set **stabilizationWindowSeconds: 30** to induce resource thrashing under fluctuating loads. When testing the autoscaling, we saw the HPA prematurely scaling down during brief traffic dips, which immediately overloaded the remaining pods (spiking CPU to 80%) and forced rapid emergency scale-ups. At peak load, the system saturates the HPA maximum, and the CPU stabilizes above the target threshold at 60–70% but can't scale further due to the **MAXPODS: 10** limit.
- Metrics Server Sensitivity:
Inherent polling delays in the Kubernetes Metrics Server, combined with CPU spikes during pod startup, can make HPA scaling seem unreliable on limited hardware.
- Pending state:
Initial CPU requests were set to 100m. At 6+ pods, this exhausted the 1-core node, leaving additional pods in a pending state. Lowering requests to 50m resolved scheduling, but means more than 6 pods hitting their 200m limit simultaneously causes heavy CPU throttling.
- The SSH tunnel (ssh -L 8080:\$CLUSTER_IP:80) is a workaround for the QEMU network boundary, not a production ingress solution.